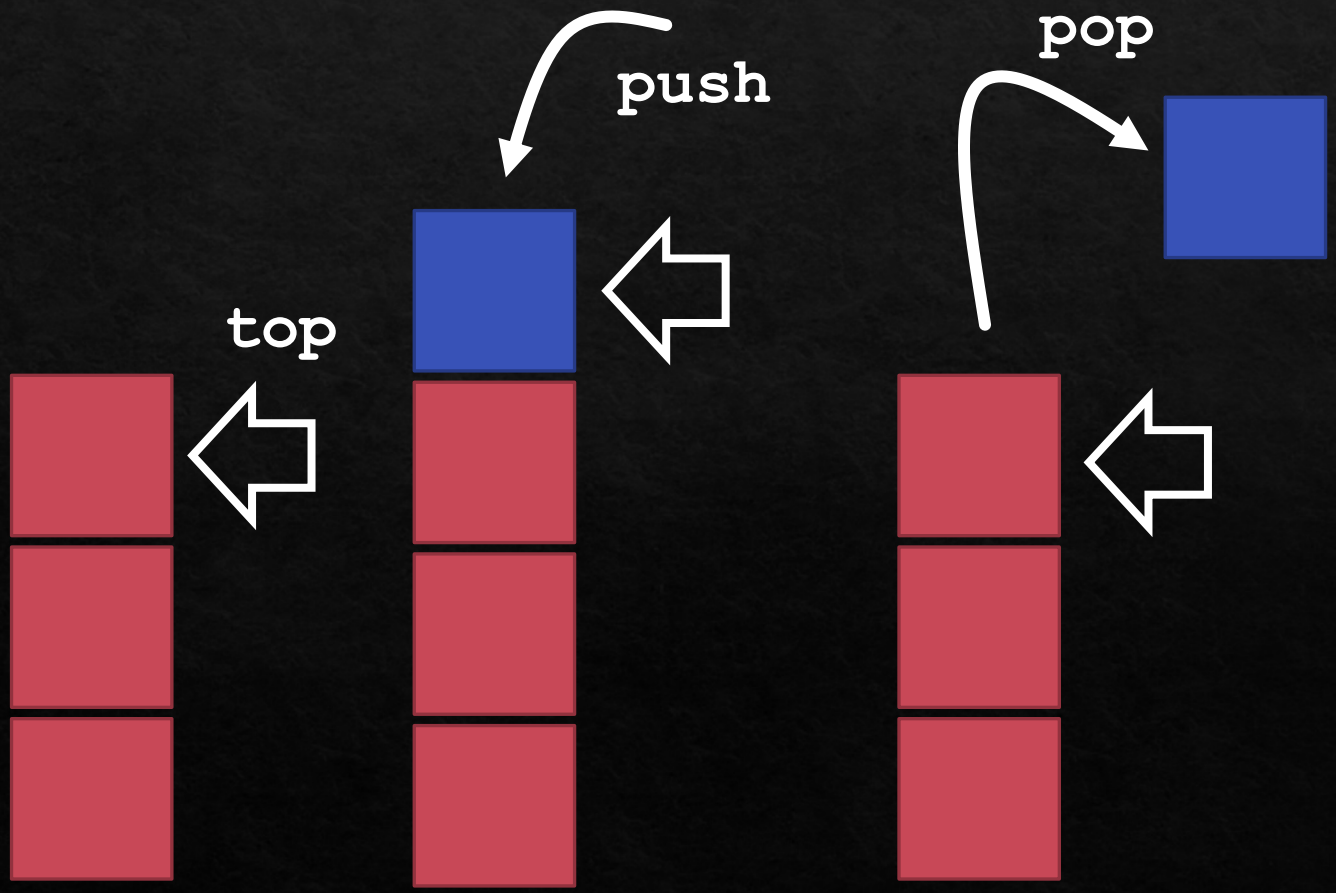
Динамические структуры данных

стек, очередь, очередь с приоритетом

Стек и очередь

Стек

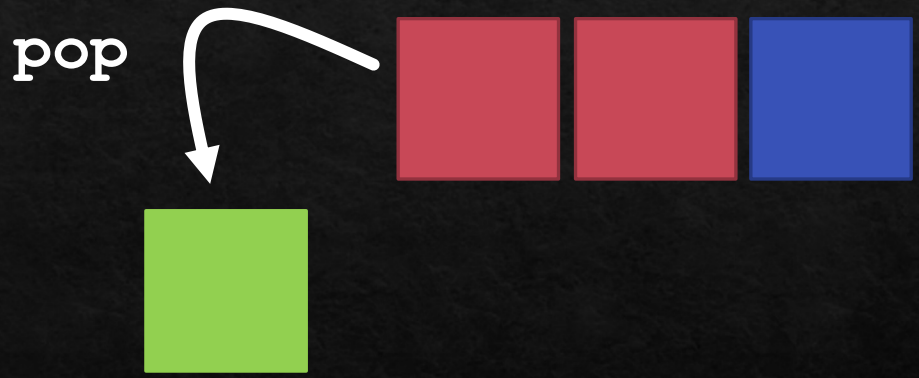
Стек – динамическая структура данных с двумя операциями: добавить (push) и удалить (pop). Стек не позволяет получить произвольный элемент из набора. Он возвращает только тот элемент, который был добавлен последним. Стек называют структурой доступа LIFO (Last In First Out).



Очередь

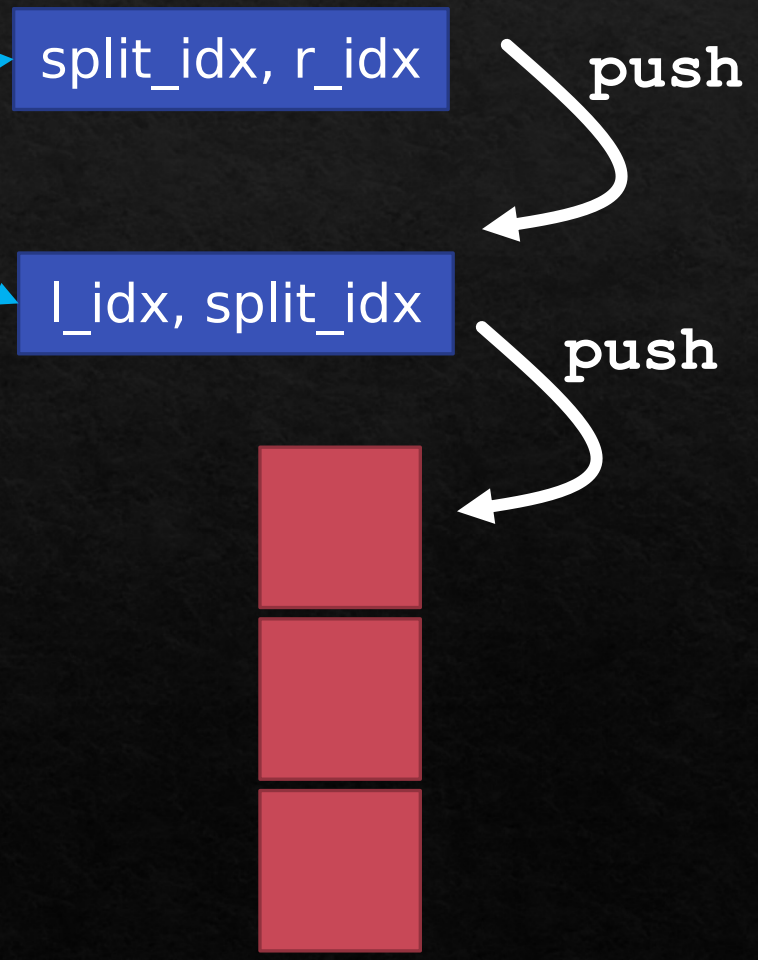
Очередь – динамическая структура данных с двумя операциями: добавить (push) и удалить (pop). Очередь не позволяет получить произвольный элемент из набора. Она возвращает только тот элемент, который был добавлен первым. Очередь называют структурой доступа FIFO (First In First Out).





Стек и очередь можно эффективно использовать для хранения промежуточных данных при преобразовании рекурсивных алгоритмов в итерационные версии.

```
23      quick_sort(data, l_idx, split_idx);
24      quick_sort(data, split_idx + 1, r_idx);
```



Идея состоит в том, чтобы сохранять пары индексов, между которыми нужно произвести партицию, в стек (или очередь). Затем в цикле обращаться к стеку (или очереди) вынимая каждую последующую пару на обработку.

Стек на базе массива

Для реализации стека с помощью массива нам понадобиться сохранять позицию вершины стека (top). При добавлении мы будем увеличивать индекс, а при удалении уменьшать. Чтобы сохранить массив и индекс вершины стека, будем использовать структуру (struct).

```
struct Stack {
    unsigned top = 0;
    int arr[100];
};
```

```
void push(Stack &s, int val) {
    s.arr[s.top++] = val;
}
```

```
int pop(Stack &s) {
    return s.arr[--s.top];
}
```

```
int main() {
    Stack s;
    push(s, 1);
    push(s, 2);
    push(s, 3);
    std::cout
        << pop(s) << ' ' << pop(s) << ' ' << pop(s)
    << std::endl;
}
```

```
3 2 1
```

Список

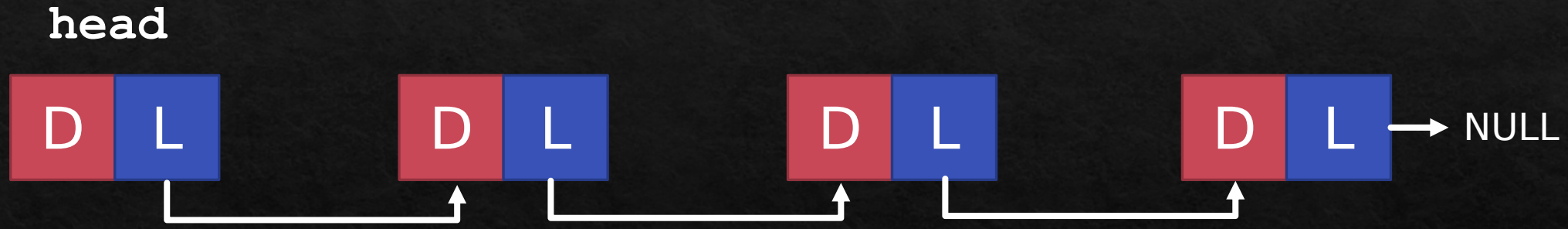
Списочная пара

Данные

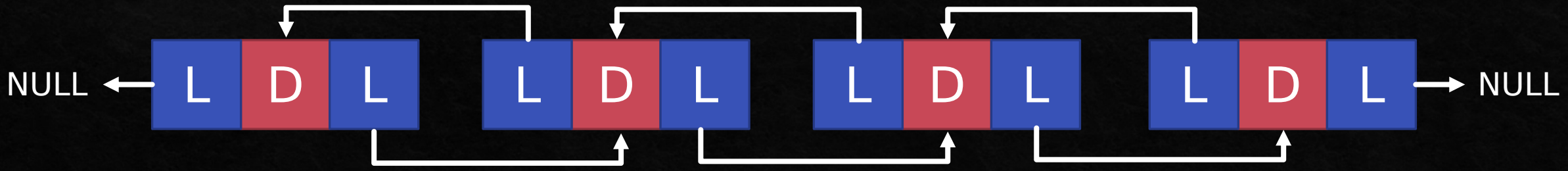


Связь со следующей списочной парой

Односвязный список



Двусвязный список



Реализация односвязного списка

```
struct ListPair {
    int data;
    ListPair *next;
};
```

```
ListPair* add_node(ListPair * const head) {
    return new ListPair {0, head};
}
```

```
ListPair* remove_node(ListPair * const head) {
    if (nullptr == head) return head;
    auto new_head = head->next;
    delete head;
    return new_head;
}
```

```
int main() {
    auto head = add_node(nullptr);
    head->data = 0;
    head = add_node(head);
    head->data = 1;
    head = add_node(head);
    head->data = 2;
    while (nullptr != head) {
        std::cout << head->data << ' ';
        head = remove_node(head);
    }
}
```

```
2 1 0
```

Реализация стека на базе списка

```
struct ListStack {
    ListPair *top = nullptr;
};
```

```
void push(ListStack &s, int val) {
    s.top = add_node(s.top);
    s.top->data = val;
}
```

```
int pop(ListStack &s) {
    auto val = s.top->data;
    s.top = remove_node(s.top);
    return val;
}
```

```
bool is_empty(ListStack const &s) {
    return s.top == nullptr;
}
```

```
int main() {
    ListStack s;
    push(s, 1);
    push(s, 2);
    push(s, 3);
    while (!is_empty(s))
        std::cout << pop(s) << ' ';

    return 0;
}
```

```
3 2 1
```

Реализация двусвязного списка

```
struct DoublyLinkedList {
    int data;
    DoublyLinkedList *next;
    DoublyLinkedList *prev;
};
```

```
DoublyLinkedList* add_next(DoublyLinkedList *tail) {
    auto new_tail = new DoublyLinkedList{0, nullptr, tail};
    if (nullptr == tail)
        return new_tail;
    return tail->next = new_tail;
}
```

```
DoublyLinkedList* add_prev(DoublyLinkedList *head) {
    auto new_head = new DoublyLinkedList{0, head, nullptr};
    if (nullptr == head)
        return new_head;
    return head->prev = new_head;
}
```

```
int main() {
    auto list = add_next(nullptr);
    list->data = 1;
    list = add_next(list);
    list->data = 2;
    list = add_next(list);
    list->data = 3;

    while (nullptr != list) {
        std::cout << list->data << ' ';
        list = list->prev;
    }

    return 0;
}
```

```
3 2 1
```

Реализация двусвязного списка

```
void remove_node(DoublyLinkedList *current) {
    if (nullptr == current) return;

    if (nullptr != current->next)
        current->next->prev = current->prev;

    if (nullptr != current->prev)
        current->prev->next = current->next;

    delete current;
}
```

```
int main() {
    auto first = add_next(nullptr);
    auto last = first;
    last->data = 1;
    last = add_next(last);
    last->data = 2;
    last = add_next(last);
    last->data = 3;

    remove_node(first->next);
    while(nullptr != first) {
        std::cout << first->data << ' ';
        first = first->next;
    }

    return 0;
}
```

```
1 3
```

Реализация очереди на базе двусвязного списка

```
struct DLLQueue {
    DoublyLinkedList *put = nullptr;
    DoublyLinkedList *get = nullptr;
};
```

```
void push(DLLQueue &q, int val) {
    q.put = add_prev(q.put);
    q.put->data = val;
    if (nullptr == q.get)
        q.get = q.put;
}
```

```
int pop(DLLQueue &q) {
    auto new_get = q.get->prev;
    auto val = q.get->data;
    remove_node(q.get);
    q.get = new_get;
    if (nullptr == q.get)
        q.put = nullptr;
    return val;
}
```

```
int main() {
    DLLQueue q;
    push(q, 1);
    push(q, 2);
    push(q, 3);

    while (!is_empty(q))
        std::cout << pop(q) << ' ';

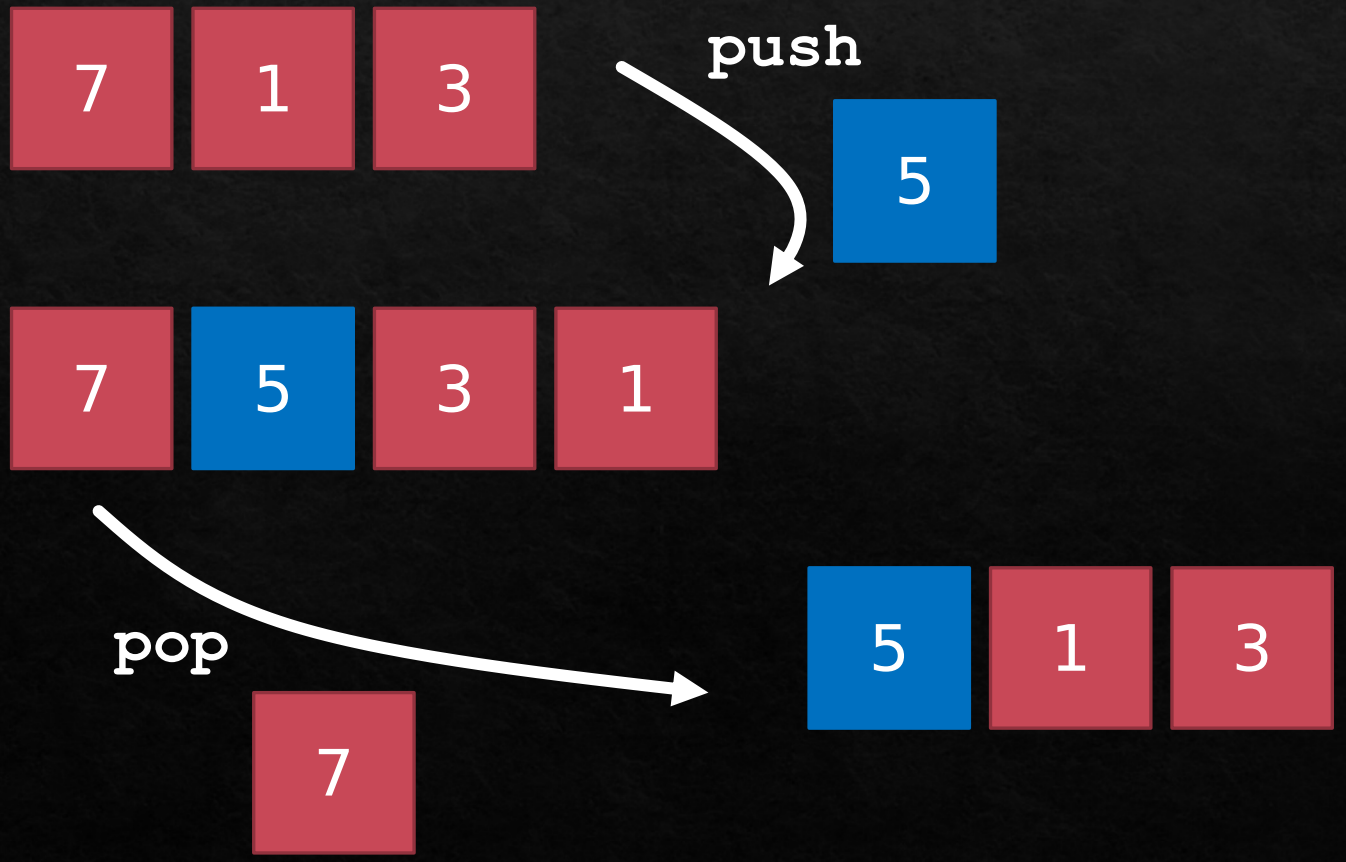
    return 0;
}
```

```
1 2 3
```

Очередь с приоритетом

Очередь с приоритетом

Очередь с приоритетом – динамическая структура данных с двумя операциями: добавить (push) и удалить (pop). Очередь с приоритетом не позволяет получить произвольный элемент из набора. Структура возвращает только тот элемент, который имеет наибольший (наименьший).



Одной из наиболее частых реализаций очереди с приоритетом является реализация на базе двоичной кучи.

Реализация очереди с приоритетом на базе max heap

```
struct PriorityQueue {
    int heap[100];
    unsigned last = 0;
};
```

```
void push(PriorityQueue &pq, int val) {
    pq.heap[pq.last++] = val;
    std::push_heap(pq.heap, pq.heap + pq.last);
}
```

```
int pop(PriorityQueue &pq) {
    int val = pq.heap[0];
    std::pop_heap(pq.heap, pq.heap + pq.last--);
    return val;
}
```

```
bool is_empty(PriorityQueue const &pq) {
    return 0 == pq.last;
}
```

```
int main() {
    PriorityQueue pq;
    push(pq, 1);
    push(pq, 2);
    push(pq, 7);
    push(pq, 5);

    while (!is_empty(pq))
        std::cout << pop(pq) << ' ';

    return 0;
}
```

```
7 5 2 1
```